

Relatório Técnico: Pipeline de Dados Massivos para Criptoativos

Disciplina: Processamento de Dados Massivos (2026/1) - PUC Minas

Autores: Pedro Henrique R. da Silva, Pedro Henrique A. de Medeiros, Marco Túlio Sousa

Repositório: <https://github.com/Pedro0204/crypto-pipeline>

1. Introdução

O mercado de criptoativos movimenta bilhões de dólares diariamente e gera um volume massivo de dados que exigem coleta contínua, processamento eficiente e organização adequada para que possam ser analisados de forma confiável. A volatilidade característica desse mercado torna necessário o acompanhamento constante de indicadores como preço, capitalização de mercado, volume de negociação e variações percentuais, exigindo infraestrutura capaz de lidar com dados em tempo real e em grande escala.

Este trabalho apresenta o desenvolvimento de um pipeline de dados completo para ingestão, processamento e visualização de dados do mercado de criptoativos. A fonte de dados utilizada é a API pública da CoinGecko, que fornece informações atualizadas sobre mais de mil criptomoedas. O objetivo principal é construir uma solução funcional que colete dados de forma contínua, aplique transformações progressivas seguindo a arquitetura Medallion (Bronze, Silver e Gold), e disponibilize os resultados em um painel interativo para análise.

O projeto foi desenvolvido utilizando tecnologias de processamento distribuído como Apache Spark e Apache Iceberg, orquestração com Apache Airflow, armazenamento em MinIO (compatível com S3) e visualização com Streamlit. Toda a infraestrutura é containerizada com Docker Compose, garantindo reprodutibilidade e facilidade de implantação. O provisionamento dos recursos de armazenamento é realizado via Terraform como infraestrutura como código.

O repositório completo do projeto está disponível no GitHub no endereço indicado acima.

2. Fonte de Dados e Caracterização

2.1 Fonte de Dados

A fonte de dados utilizada é a API REST pública da CoinGecko, especificamente o endpoint `/coins/markets`. Esse endpoint retorna dados de mercado atualizados das principais criptomoedas, incluindo preços em dólar americano, capitalização de mercado, volume de

negociação nas últimas 24 horas e variações de preço.

A coleta é feita em ciclos de 5 páginas com 250 moedas por página, totalizando até 1.250 criptomoedas por ciclo de ingestão. Os dados são coletados a cada 30 segundos de forma contínua pelo módulo de streaming.

2.2 Caracterização dos Dados

Cada registro coletado da API contém os seguintes campos principais:

Campo	Tipo	Descrição
id	String	Identificador único da moeda (ex: bitcoin, ethereum)
symbol	String	Símbolo da moeda (ex: btc, eth)
name	String	Nome completo da moeda
current_price	Double	Preço atual em dólares
market_cap	Double	Capitalização de mercado
total_volume	Double	Volume total de negociação em 24h
high_24h	Double	Maior preço nas últimas 24h
low_24h	Double	Menor preço nas últimas 24h
price_change_24h	Double	Variação absoluta de preço em 24h
price_change_percentage_24h	Double	Variação percentual de preço em 24h
market_cap_rank	Integer	Posição no ranking por capitalização

Além desses campos originais, o pipeline adiciona metadados de controle: `ingestion_ts` (timestamp da coleta), `dt` (data no formato AAAA-MM-DD) e `hour` (hora da coleta). Esses campos são utilizados para particionamento e rastreamento dos dados ao longo do pipeline.

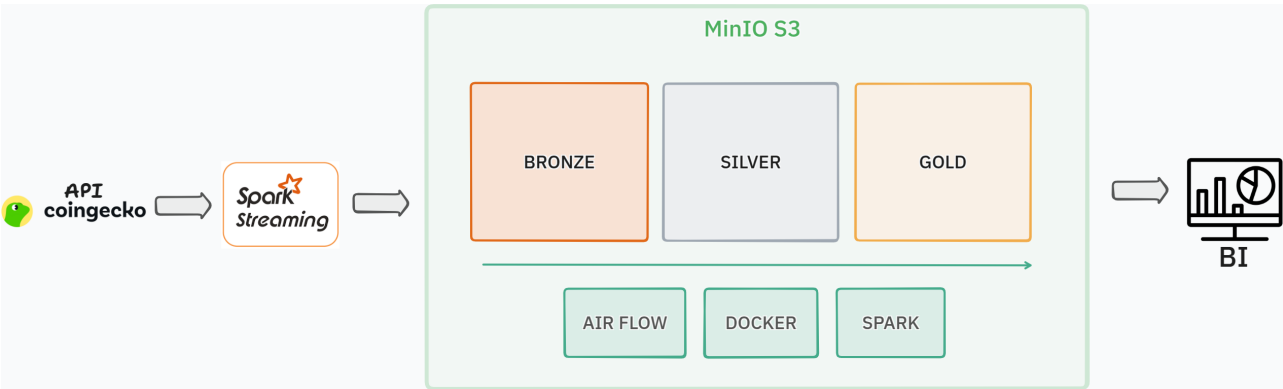
Os dados apresentam características típicas de séries temporais financeiras: alta frequência de atualização, valores numéricos com grande amplitude (preços variam de frações de centavo a dezenas de milhares de dólares) e presença de valores nulos em moedas com menor liquidez.

3. Arquitetura da Solução

3.1 Visão Geral

A arquitetura segue o padrão Medallion, organizado em três camadas de dados (Bronze, Silver e Gold), cada uma com nível crescente de refinamento e qualidade. Todo o ecossistema é orquestrado por Docker Compose com comunicação entre containers por rede interna.

3.2 Diagrama de Arquitetura



O diagrama ilustra o fluxo de dados desde a API da CoinGecko até o painel de BI. Os dados são coletados pelo Spark Streaming e armazenados no MinIO S3, passando pelas camadas Bronze, Silver e Gold. A infraestrutura de suporte (Airflow, Docker e Spark) é representada na parte inferior.

3.3 Componentes da Arquitetura

Componente	Tecnologia	Função
Ingestão	Spark Structured Streaming	Coleta contínua da API CoinGecko
Object Storage	MinIO (compatível com S3)	Armazenamento dos dados em buckets

Formato de Tabela	Apache Iceberg	Tabelas com garantia ACID e versionamento
Processamento	Apache Spark 3.5.3	Transformações e agregações em lote
Orquestração	Apache Airflow 2.10.5	Agendamento e monitoramento do pipeline
Dashboard	Streamlit + Plotly	Visualização interativa dos dados
Infraestrutura	Docker Compose	Containerização de todos os serviços
IaC	Terraform	Provisionamento automático dos buckets
Fonte de Dados	CoinGecko API	Dados de mercado de criptoativos em tempo real

3.4 Camadas do Medallion

Camada	Formato	Particionamento	Descrição
Bronze	JSON	dt e hour	Dados brutos da API, append-only, sem transformação
Silver	Apache Iceberg	dt	Dados tipados, deduplicados, com controle de qualidade
Gold	Apache Iceberg	dt	Star Schema com tabela fato e dimensão, pronto para análise

3.5 Infraestrutura Docker

A infraestrutura é composta por sete serviços Docker orquestrados via Docker Compose:

- **MinIO:** armazenamento de objetos compatível com S3, com três buckets (bronze, silver, gold).
- **Spark Master e Worker:** cluster Spark para execução dos jobs de processamento, com imagem customizada contendo as bibliotecas do Iceberg e conectores S3A/Hadoop.
- **Airflow (Webserver, Scheduler e PostgreSQL):** orquestração do pipeline com banco de metadados PostgreSQL e executor local.
- **Airflow Init:** serviço de inicialização que executa a migração do banco e cria o usuário administrador.

A imagem Spark customizada inclui os JARs do Iceberg (iceberg-spark-runtime-3.5_2.12-1.7.1), Hadoop AWS (hadoop-aws-3.3.4) e AWS SDK (aws-java-sdk-bundle-1.12.262). A imagem Airflow customizada inclui o provider Apache Spark para o operador `SparkSubmitOperator`.

Todos os serviços estão conectados pela rede Docker `crypto-net`, permitindo comunicação interna por nomes de container. Os volumes persistentes garantem que os dados do MinIO e do PostgreSQL sobrevivam a reinicializações dos containers.

3.6 Estrutura do Projeto

A organização dos arquivos do projeto segue a separação por responsabilidade, com diretórios dedicados para infraestrutura, código-fonte e orquestração:

```
crypto-pipeline/
├── docker-compose.yml      # MinIO + Spark + Airflow
├── .env                    # Variáveis de ambiente
├── pyproject.toml          # Dependências Python
├── run_all.py              # Automação de inicialização
├── infra/
│   ├── docker/
│   │   ├── spark/Dockerfile # Spark + Iceberg JARs
│   │   └── airflow/Dockerfile # Airflow + Spark provider
│   └── terraform/           # Buckets bronze/silver/gold (IaC)
├── src/
│   ├── ingestao/
│   │   ├── coingecko_producer.py # Coletor standalone da API
│   │   └── streaming/
│   │       ├── spark_bronze.py    # Spark Streaming -> Bronze
│   │       └── processamento/
│   │           ├── silver/
│   │           │   ├── mercado_silver.py # Bronze -> Silver (Iceberg)
│   │           │   └── gold/
│   │           │       ├── metrics_gold.py # Silver -> Gold (Star Schema)
│   │           │       └── maintenance/
│   │           │           ├── compaction.py # OPTIMIZE diário
│   │           │           └── vacuum.py      # Expire snapshots 7d
│   │           └── dashboard/
│   │               └── app.py          # Streamlit BI
```

```
■■■ airflow/
    ■■■ dags/
        ■■■ crypto_pipeline_dag.py # DAG: Silver -> Gold -> Compact -> Vacuum
```

3.7 Serviços e URLs

Todos os serviços do pipeline são acessíveis via navegador após a inicialização com `docker compose up -d`:

Serviço	URL	Credenciais
MinIO Console	http://localhost:9001	minioadmin / minioadmin
Spark Master UI	http://localhost:8085	(sem autenticação)
Apache Airflow	http://localhost:8080	admin / admin
Streamlit Dashboard	http://localhost:8501	(sem autenticação)

4. Explicação Detalhada do Pipeline

4.1 Ingestão (Camada Bronze)

O processo de ingestão é executado pelo módulo `spark_bronze.py` utilizando Spark Structured Streaming. O funcionamento segue o padrão `foreachBatch`, onde um rate source dispara a cada 30 segundos e, a cada disparo, uma função de callback realiza a chamada à API da CoinGecko.

O processo de cada lote segue os passos:

1. A trigger do rate source ativa o processamento.
2. O módulo realiza chamadas HTTP ao endpoint `/coins/markets` da CoinGecko, coletando 5 páginas de 250 moedas cada.
3. Cada registro recebe os metadados `ingestion_ts`, `dt` e `hour`.
4. Os dados são gravados em formato JSON no bucket Bronze do MinIO, particionados por `dt` e `hour`.

A ingestão é limitada a um único worker do Spark para respeitar o rate limit da API da CoinGecko (30 requisições por segundo no plano gratuito). Essa decisão garante que não ocorram bloqueios por excesso de requisições.

Os detalhes técnicos da ingestão são resumidos a seguir:

Parâmetro	Valor
Arquivo	<code>src/streaming/spark_bronze.py</code>
Modo	Spark Structured Streaming
Trigger	Rate source (1 requisição a cada 30s)
Destino	MinIO bucket <code>bronze/</code>
Formato	JSON (append-only)
Particionamento	<code>dt=AAAA-MM-DD / hour=HH</code>
Workers	<code>local[1]</code> (rate limit safe)

Além do módulo de streaming, o projeto conta com o produtor standalone `src/ingestao/coingecko_producer.py`, que encapsula a interação com a API da CoinGecko e pode ser utilizado para testes de coleta de forma independente.

4.2 Processamento Silver

O processamento Silver é executado diariamente pelo módulo `mercado_silver.py`, acionado pelo Airflow. O objetivo dessa etapa é transformar os dados brutos do Bronze em uma tabela estruturada e confiável.

As transformações aplicadas são:

1. **Leitura:** os arquivos JSON do Bronze são lidos a partir do caminho correspondente à data de execução (`dt={execution_date}`).
2. **Cast de tipos:** os campos numéricos recebidos como strings são convertidos para os tipos adequados (double e integer).
3. **Conversão de timestamp:** o campo `ingestion_ts` é convertido de string para o tipo timestamp do Spark.
4. **Adição de metadados:** o campo `_processed_at` é adicionado com o horário do processamento.
5. **Deduplicação:** é aplicada uma janela (window function) particionada por (`id, dt, hour`) e ordenada por `ingestion_ts` de forma decrescente. Apenas o registro mais recente de cada combinação de moeda, data e hora é mantido.

6. **Gravação:** os dados deduplicados são gravados na tabela Iceberg `iceberg.crypto.coins_markets_silver`, particionada por `dt`.

A utilização do Apache Iceberg nessa camada garante transações ACID, versionamento de snapshots e compatibilidade com o protocolo S3A do MinIO.

4.3 Processamento Gold

O processamento Gold é executado diariamente após a conclusão da etapa Silver, pelo módulo `metricas_gold.py`. Nessa etapa, os dados são modelados em Star Schema com duas tabelas.

Tabela Fato (`fct_metricas_hora`):

Essa tabela contém agregações horárias agrupadas por (`id`, `symbol`, `name`, `dt`, `hour`). As métricas calculadas incluem:

Métrica	Descrição
<code>total_registros</code>	Quantidade de registros na hora
<code>preco_medio</code>	Preço médio
<code>preco_min</code>	Preço mínimo
<code>preco_max</code>	Preço máximo
<code>preco_desvio</code>	Desvio padrão do preço
<code>market_cap_medio</code>	Capitalização de mercado média
<code>volume_medio</code>	Volume médio de negociação
<code>variacao_24h_media</code>	Variação absoluta média em 24h
<code>variacao_pct_24h_media</code>	Variação percentual média em 24h
<code>high_24h</code>	Maior preço em 24h (máximo do grupo)
<code>low_24h</code>	Menor preço em 24h (mínimo do grupo)
<code>spread_24h</code>	Diferença entre high e low em 24h

`volatilidade_relativa`

Desvio padrão dividido pelo preço médio, em percentual

Tabela Dimensão (`dim_moedas`):

Contém o snapshot diário de cada moeda. Uma window function particionada por `id` e ordenada por `ingestion_ts` de forma decrescente seleciona o registro mais recente de cada moeda no dia. Os campos incluem identificação (`id`, `symbol`, `name`), ranking por capitalização, preço atual, capitalização, volume e variações.

Ambas as tabelas são gravadas em formato Iceberg no bucket Gold do MinIO.

4.4 Manutenção (Compactação e Vacuum)

O pipeline inclui duas tarefas de manutenção para garantir desempenho e controle de espaço em disco.

Compactação (diária): executada pelo módulo `compaction.py` após o processamento Gold. Utiliza o procedimento `rewrite_data_files` do Iceberg para consolidar arquivos pequenos em arquivos maiores, otimizando a leitura e reduzindo a fragmentação nas tabelas Silver e Gold.

Vacuum (semanal): executado aos domingos pelo módulo `vacuum.py`. Realiza duas operações por tabela: `expire_snapshots` remove snapshots com mais de 7 dias, e `remove_orphan_files` elimina arquivos que não estão referenciados por nenhum snapshot. Essa manutenção evita o crescimento descontrolado do armazenamento.

4.5 Orquestração com Airflow

O pipeline completo é orquestrado por uma DAG do Airflow (`crypto_pipeline_dag.py`) com execução diária. A sequência de tarefas é:

```
silver_etl → gold_etl → compaction → check_vacuum_day → [vacuum | skip_vacuum] → end
```

Cada tarefa de processamento utiliza o operador `SparkSubmitOperator`, que submete os jobs ao cluster Spark. A task `check_vacuum_day` é um `BranchPythonOperator` que verifica se o dia de execução é domingo: caso positivo, executa o `vacuum`; caso contrário, segue para o encerramento.

A DAG possui as seguintes configurações:

- **Agendamento:** diário (meia-noite UTC)
- **Retentativas:** 2 tentativas com intervalo de 5 minutos
- **Catchup:** desabilitado (não executa datas retroativas)
- **Data de início:** 1 de junho de 2026

4.6 Dashboard

O painel de visualização foi desenvolvido com Streamlit e Plotly, acessível na porta 8501. O dashboard se conecta ao MinIO via boto3 e lê os dados Parquet do bucket Gold.

As visualizações disponíveis são:

- **KPIs principais:** total de moedas rastreadas, capitalização de mercado total, volume total em 24h e maior variação percentual em 24h.
- **Top N por capitalização de mercado:** gráfico de barras horizontais com escala de cores baseada na variação percentual de 24h, com slider para ajustar o número de moedas exibidas (10 a 100).
- **Variação de preço em 24h:** gráfico de barras com as moedas ordenadas por variação percentual.
- **Spread 24h:** gráfico das 20 moedas com maior diferença entre o preço máximo e mínimo em 24h.
- **Métricas horárias:** quando disponíveis, exibe a tendência de preço (médio, máximo e mínimo), histograma de volume por hora e gráfico de volatilidade relativa, com seletor de moeda.

5. Problemas Encontrados e Soluções

5.1 Rate Limit da API CoinGecko

Problema: A API da CoinGecko no plano gratuito impõe um limite de 30 requisições por segundo. Em testes iniciais, a execução com múltiplos workers do Spark gerava requisições paralelas que excediam esse limite, resultando em respostas com código HTTP 429 (Too Many Requests).

Solução: A ingestão foi configurada para utilizar apenas um worker do Spark (`local[1]`) e o intervalo entre lotes foi definido em 30 segundos via rate source. Essa abordagem sequencializa as chamadas à API e respeita o limite imposto.

5.2 Dados Duplicados na Camada Bronze

Problema: Como a coleta é contínua e a API pode retornar os mesmos dados em chamadas próximas (especialmente quando os preços não se alteraram), a camada Bronze acumula registros duplicados. Esses duplicados, se não tratados, distorceriam as agregações na camada Gold.

Solução: A camada Silver implementa deduplicação por meio de window functions particionadas por (`id`, `dt`, `hour`). Apenas o registro mais recente de cada moeda por hora é mantido,

eliminando redundâncias sem perda de informação relevante.

5.3 Fragmentação de Arquivos Pequenos

Problema: O Spark Structured Streaming, ao gravar dados a cada 30 segundos no formato JSON, produz um grande número de arquivos pequenos no bucket Bronze. Esse padrão impacta negativamente o desempenho de leitura nas etapas seguintes do pipeline, pois cada arquivo pequeno gera overhead de abertura e metadados.

Solução: A tarefa de compactação diária utiliza o procedimento `rewrite_data_files` do Iceberg para consolidar os arquivos pequenos das tabelas Silver e Gold em arquivos maiores e otimizados. Dessa forma, o impacto de desempenho é mitigado sem alterar a estratégia de ingestão.

5.4 Crescimento de Snapshots do Iceberg

Problema: Cada operação de escrita no Iceberg gera um novo snapshot, o que ao longo do tempo consome espaço de armazenamento e pode degradar a performance de consultas que precisam listar o histórico de versões.

Solução: A tarefa de vacuum semanal remove snapshots com mais de 7 dias e elimina arquivos órfãos do armazenamento. Essa rotina mantém o espaço em disco controlado sem comprometer a capacidade de recuperação de dados recentes.

5.5 Compatibilidade MinIO com Protocolo S3

Problema: O MinIO é compatível com a API S3, porém requer configurações específicas que diferem do AWS S3. Em particular, o acesso via path-style (em vez de virtual-hosted-style) é obrigatório, e a validação de credenciais padrão do SDK AWS precisa ser desabilitada.

Solução: A configuração do Spark e do Terraform foi ajustada para utilizar `path.style.access = true`, desabilitar a validação de credenciais AWS e apontar o endpoint diretamente para o MinIO. No Terraform, o provider AWS foi configurado com os parâmetros `skip_credentials_validation`, `skip_metadata_api_check` e `skip_requesting_account_id` para evitar chamadas à infraestrutura AWS real.

6. Resultados Obtidos e Conclusões

6.1 Resultados

Os principais indicadores da solução são:

Indicador	Valor
Camadas Medallion	3 (Bronze, Silver, Gold)
Intervalo de ingestão	30 segundos
Retenção de snapshots	7 dias
Serviços containerizados	7 (MinIO, Spark Master, Spark Worker, Airflow Webserver, Airflow Scheduler, Airflow PostgreSQL, Airflow Init)

O pipeline desenvolvido alcançou os seguintes resultados:

- **Ingestão contínua** de dados de até 1.250 criptomoedas a cada 30 segundos, com coleta ininterrupta operando em background via Spark Structured Streaming.
- **Organização em camadas** seguindo a arquitetura Medallion, garantindo rastreabilidade dos dados desde a coleta bruta (Bronze) até as métricas prontas para análise (Gold).
- **Deduplicação eficiente** na camada Silver, eliminando registros redundantes e assegurando a confiabilidade dos dados utilizados nas agregações.
- **Modelagem Star Schema** na camada Gold, com tabela fato contendo métricas horárias (preço médio, volatilidade, volume, spread) e tabela dimensão com o perfil atualizado de cada moeda.
- **Orquestração automatizada** via Airflow, com pipeline diário completo (Silver, Gold, compactação) e manutenção semanal (vacuum), incluindo tratamento de falhas com retentativas.
- **Painel interativo** com Streamlit e Plotly para visualização de KPIs, rankings por capitalização, variações de preço e métricas horárias por moeda.
- **Infraestrutura reprodutível** com Docker Compose e provisionamento automatizado de buckets via Terraform.

A execução do pipeline foi validada com sucesso. A DAG do Airflow completou todas as etapas (Silver ETL, Gold ETL, compactação e vacuum) com status de sucesso, conforme verificado na interface web do Airflow. O dashboard Streamlit apresentou corretamente os dados processados, com os gráficos e KPIs operacionais.

6.2 Conclusões

O projeto demonstrou a viabilidade de construir um pipeline de dados massivos funcional e automatizado para o domínio de criptoativos utilizando ferramentas de código aberto. A arquitetura Medallion provou ser adequada para organizar dados em estágios progressivos de

qualidade, facilitando tanto a manutenção quanto a evolução do pipeline.

A combinação de Spark Structured Streaming para ingestão contínua e Spark em lote para transformações ofereceu flexibilidade para lidar com os diferentes requisitos de cada camada. O Apache Iceberg se mostrou uma escolha apropriada como formato de tabela, fornecendo garantias ACID, versionamento por snapshots e procedimentos de manutenção integrados que simplificaram tarefas como compactação e limpeza de dados antigos.

A containerização com Docker Compose garantiu que toda a infraestrutura pudesse ser reproduzida em qualquer ambiente com um único comando. O uso de Terraform para provisionamento dos buckets adicionou uma camada de infraestrutura como código que complementa a automação do projeto.

Como possíveis evoluções, destacam-se a substituição do executor local do Airflow por CeleryExecutor para suportar paralelismo de tarefas, a adição de monitoramento de qualidade de dados com ferramentas como Great Expectations, e a expansão do dashboard com funcionalidades de alertas automáticos para variações significativas de mercado.